B28: SQL SuperHero Program : Session 42

1 message

Vishal Jaiswal <vishaldbdev@gmail.com> 4 May 2025 at 06:11
Bcc: satishhattekar1997@gmail.com

When we run any pl/sql block we need to use the data types as placeholder to hold data from the select statement. Does not matter , you are running simple select queries or writing a bulk collect program. Both behave the same way. We need an INTO clause in a select statement.
Datatype : Basically 2 types.

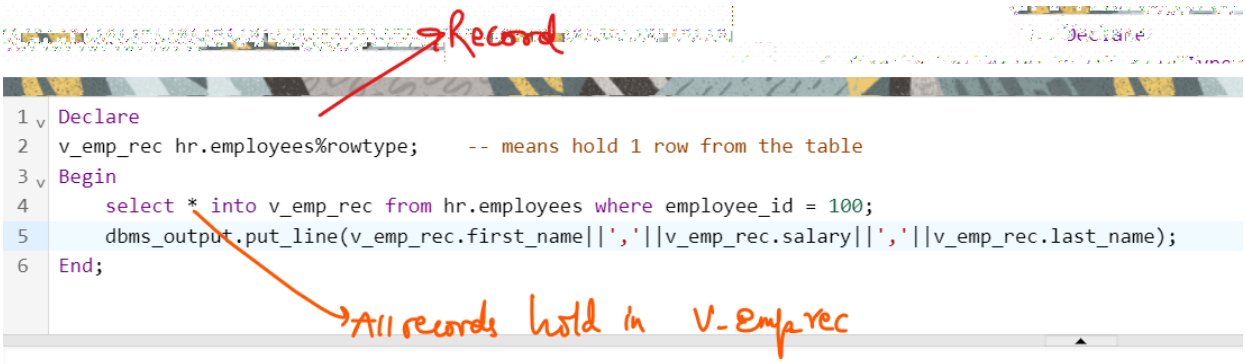
- Scalar ----------> Only 1 Value
- Composite ----------> More than 1 Value

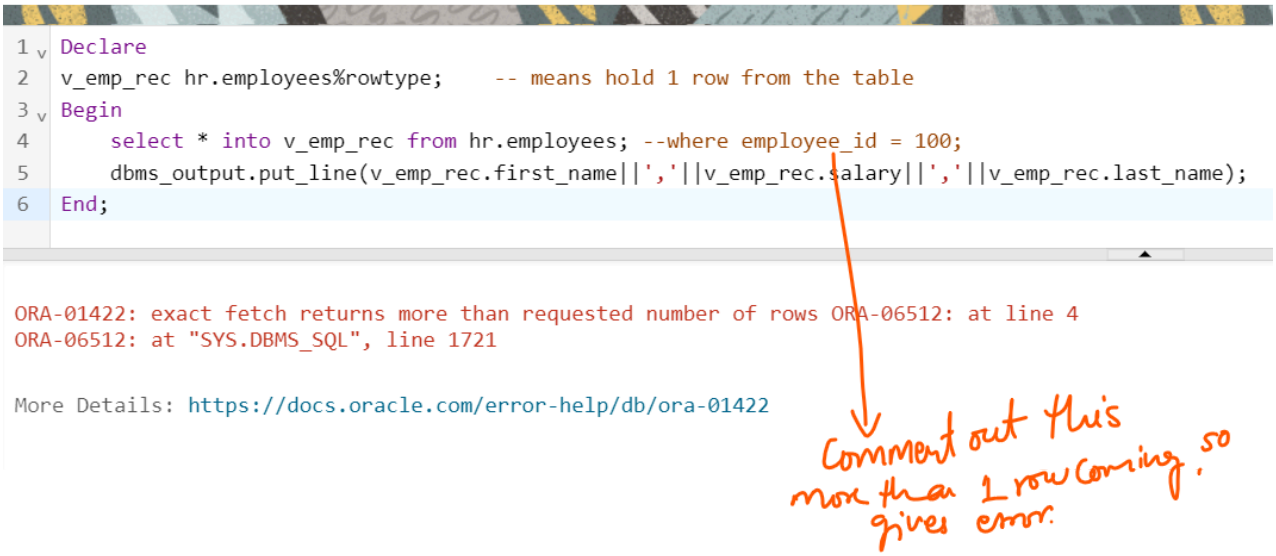
1. **Record**
   - Type
   - %RowType
2. **Collection**
   - Varray
   - Nested Table
   - Associative Array (PL/SQL Table OR Index By Table)

```
1  Declare
2      L_Name Hr.Employees.First_name%Type;
3      L_Sal Hr.Employees.Salary%Type;
4  Begin
5      Select First_name, Salary Into L_Name, L_Sal
6      From Hr.Employees Where Employee_Id = 100;
7  End;
```

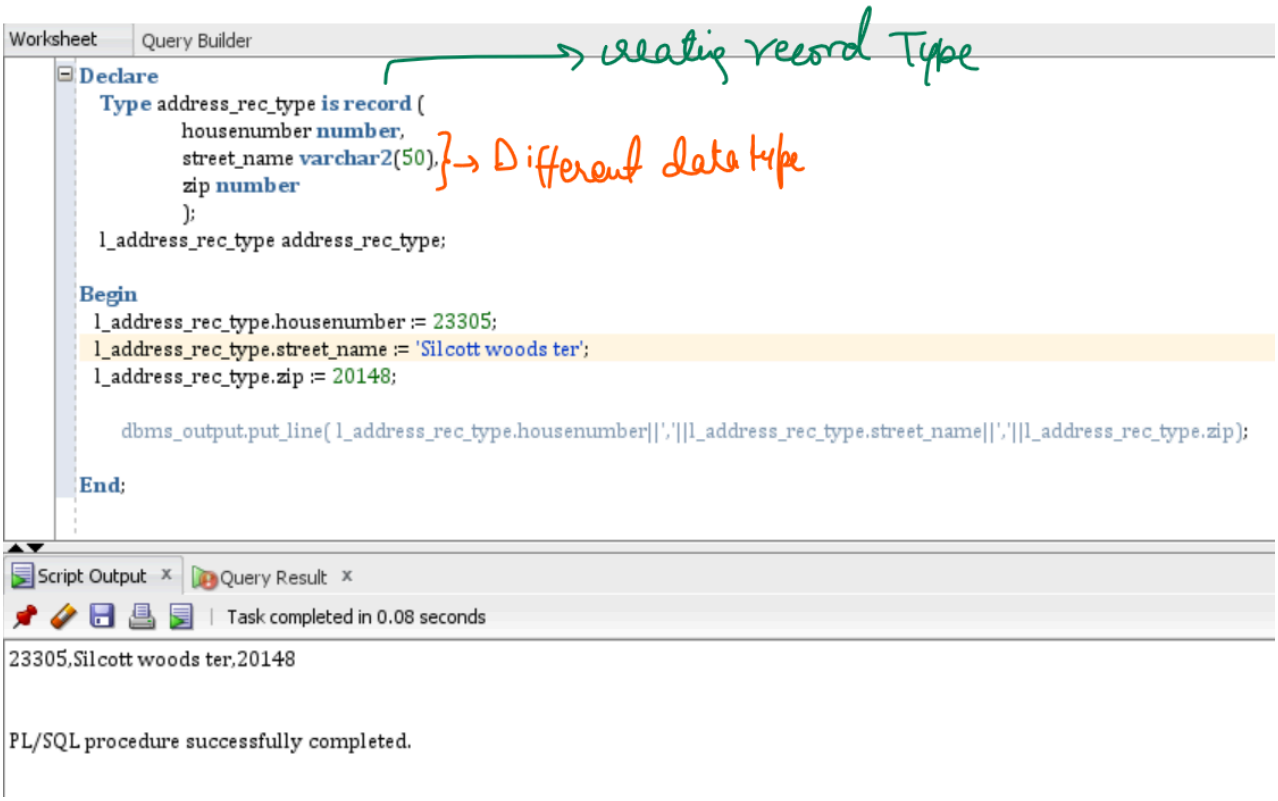
→ Scalar data type
↓
Number
Varchar
date
Timestamp
clob
etc.....

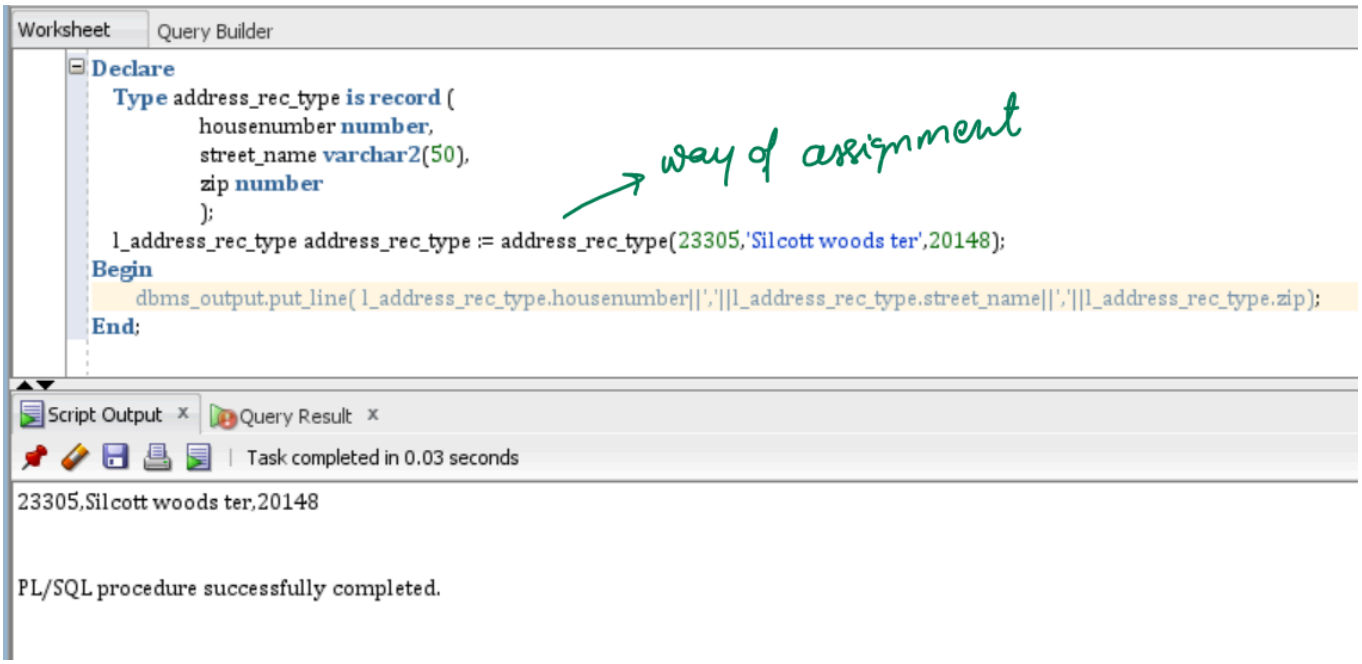
%Rowtype use as RECORD





TYPE use as RECORD





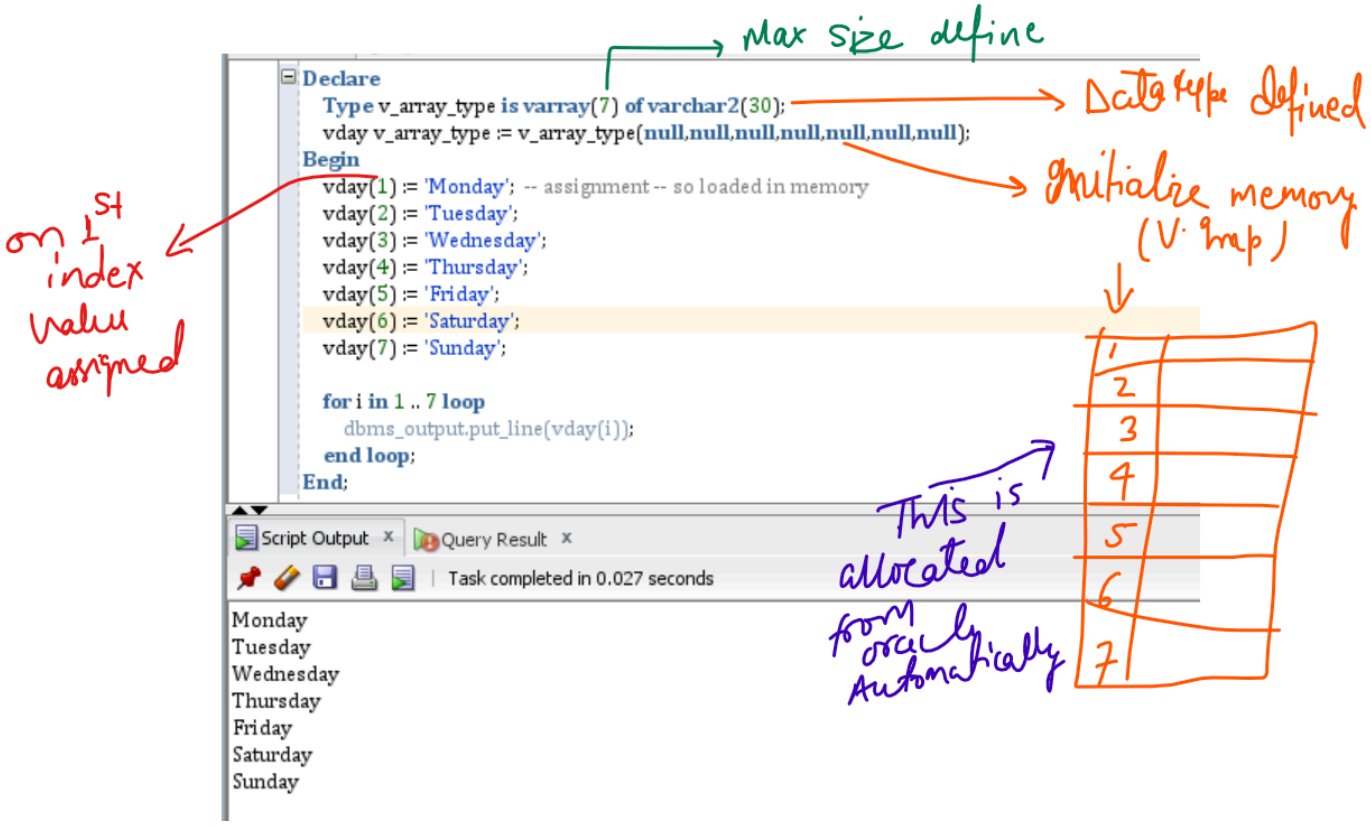
- Collection can hold ordered groups of logically related elements
- In a collection internal components always have the same datatype.(Collections are homogeneous)
- In a Record internal components always have different data types.()
- You can dynamically resize collections using methods like EXTEND for PL/SQL tables or EXTEND and TRIM for nested tables, but Records have a fixed structure defined at compile time, and you cannot change the number or data types of fields after declaration.
- Elements in a collection (e.g., elements in a PL/SQL table or elements in a nested table) are accessed by their index. You use the subscript notation (e.g., my_collection(1)) to access individual elements but Fields in a record are accessed by their names. You use dot notation (e.g., my_record.field_name) to access individual fields within the record.

==========
==========

Varray

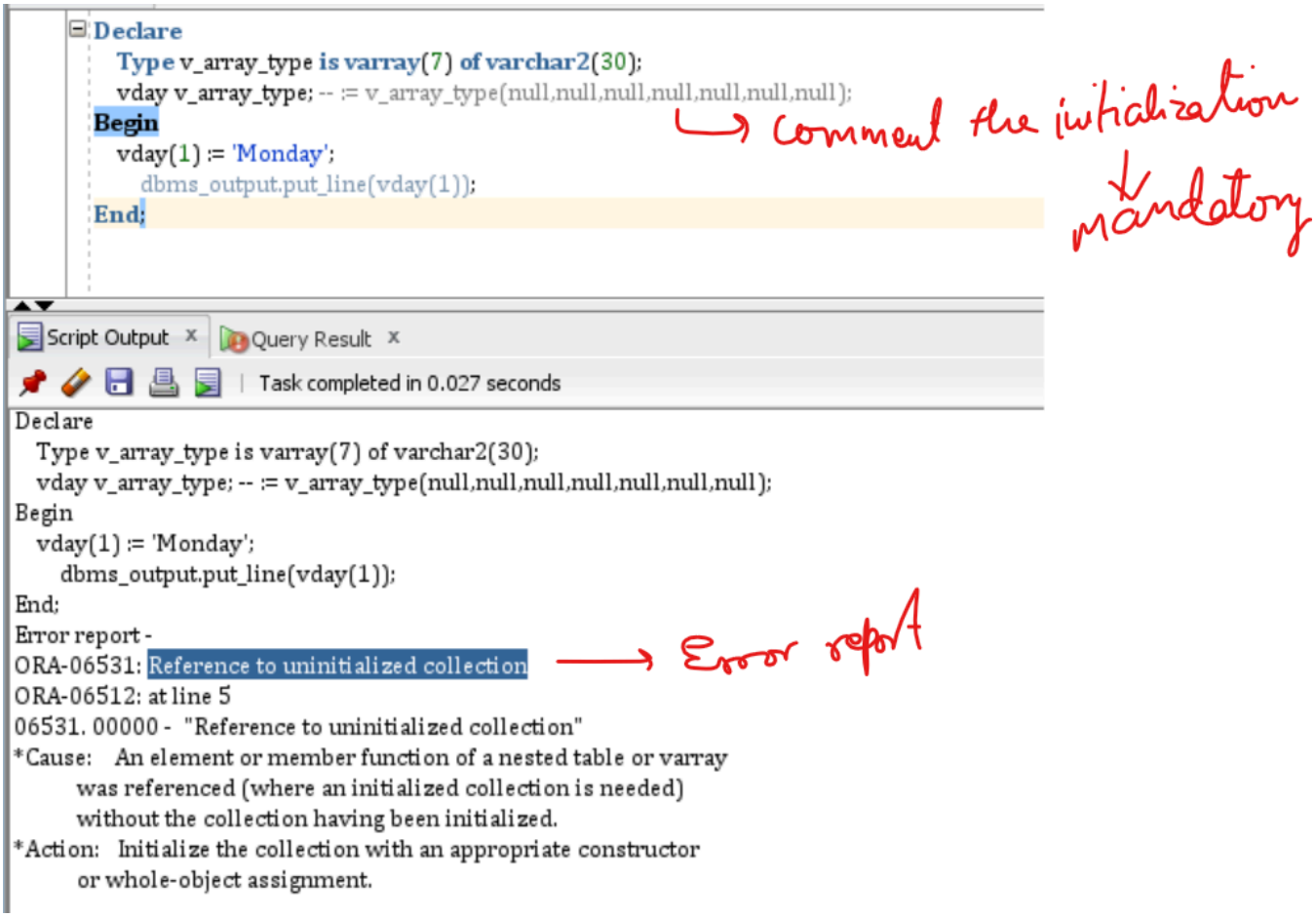
- A Type of collection. Varray is like just a simple array of any programming language. you can call a variable array.
- VARRAY, "Variable-Size Array," is a composite data type that allows you to create an ordered collection of elements of the same data type with a fixed maximum size.
- A VARRAY has a fixed maximum size that is specified when you define the type. Number of elements can vary from 0 to max size
- It must have a max element limit
- VARRAY elements are ordered and can be accessed by their position (index) within the array
- You can not delete random elements from varray . Either delete from last using **TRIM** function or delete the entire array using **DELETE**

```
Declare
   Type v_array_type is varray(7) of varchar2(30);
   vday v_array_type := v_array_type(null,null,null,null,null,null,null);
Begin
   vday(1) := 'Monday';  -- assignment -- so loaded in memory
   vday(2) := 'Tuesday';
   vday(3) := 'Wednesday';
   vday(4) := 'Thursday';
   vday(5) := 'Friday';
   vday(6) := 'Saturday';
   vday(7) := 'Sunday';

   for i in 1 .. 7 loop
      dbms_output.put_line(vday(i));
   end loop;
End;
```

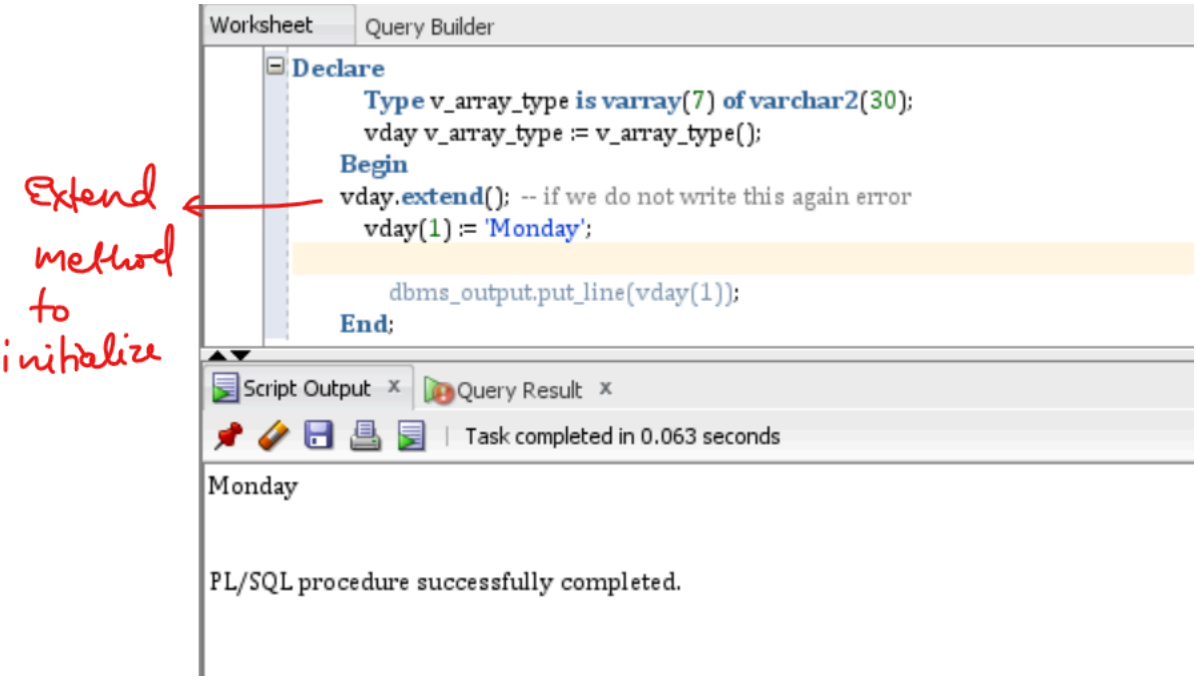


When we work on collection 3 types of error we encountered :

1. **Reference to uninitialized collection**



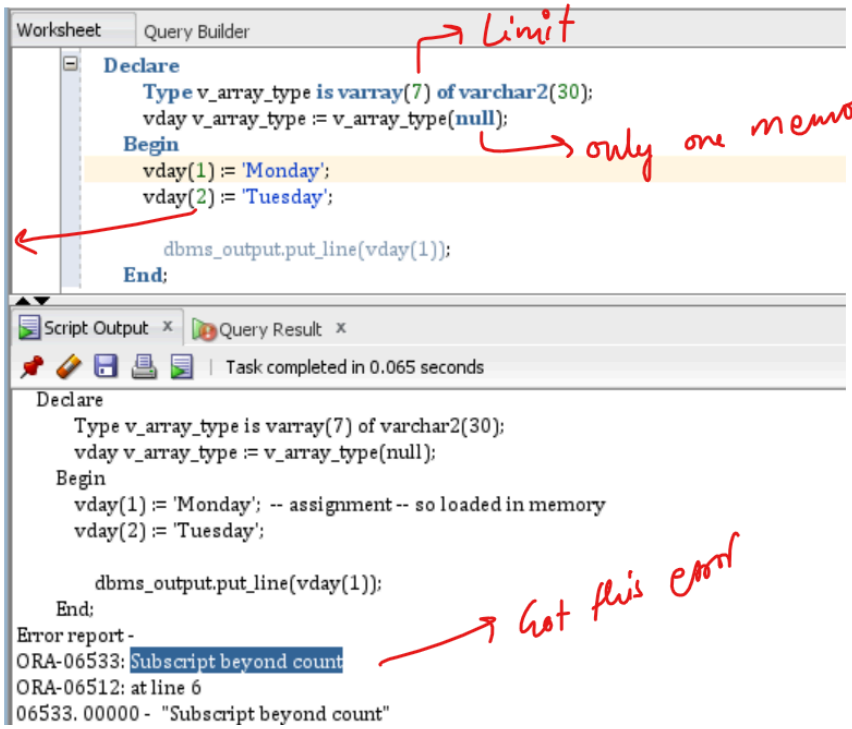
It is mandatory to initialize, else we got this error. We either have to initialize in declare section or use **EXTEND** Method.



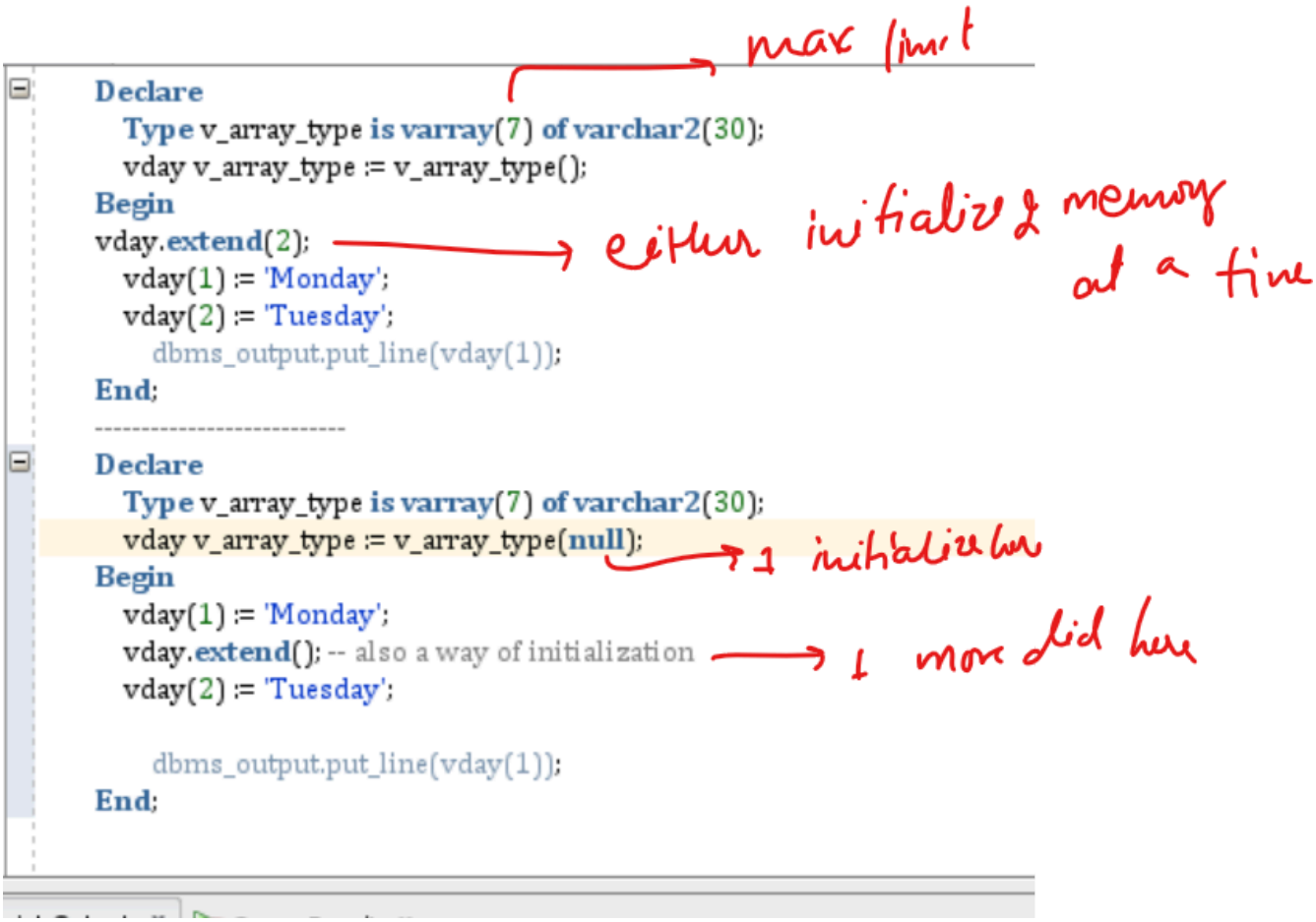
2. **Subscript beyond count**

```
Declare
Type v_array_type is varray(7) of varchar2(30);
vday v_array_type := v_array_type(null);
Begin
vday(1) := 'Monday';
```

```
vday(2) := 'Tuesday';

dbms_output.put_line(vday(1));
End;
```




```
 Declare
Type v_array_type is varray(7) of varchar2(30);
vday v_array_type := v_array_type();
Begin
      vday.extend(2);
vday(1) := 'Monday';
vday(2) := 'Tuesday';
dbms_output.put_line(vday(1));
End;

---------------------------

Declare
Type v_array_type is varray(7) of varchar2(30);
vday v_array_type := v_array_type(null);
Begin
vday(1) := 'Monday';
vday.extend(); -- also a way of initialization
vday(2) := 'Tuesday';

dbms_output.put_line(vday(1));
End;
```




```
Declare
  Type v_array_type is varray(7) of varchar2(30);
  vday v_array_type := v_array_type();
Begin
vday.extend(2);
  vday(1) := 'Monday';
  vday(2) := 'Tuesday';
    dbms_output.put_line(vday(1));
End;
-------------------------
Declare
  Type v_array_type is varray(7) of varchar2(30);
  vday v_array_type := v_array_type(null);
Begin
  vday(1) := 'Monday';
  vday.extend(); -- also a way of initialization
  vday(2) := 'Tuesday';

    dbms_output.put_line(vday(1));
End;
```

3. **subscription outside limit**

```
 Declare
Type v_array_type is varray(7) of varchar2(30);
vday v_array_type := v_array_type();
Begin
      vday.extend(8);
vday(1) := 'Monday';
vday(2) := 'Tuesday';
dbms_output.put_line(vday(1));
End;
```

limit

```
Declare
Type v_array_type is varray(7) of varchar2(30);
vday v_array_type := v_array_type();
Begin
     vday.extend(8);
vday(1) := 'Monday';
vday(2) := 'Tuesday';
dbms_output.put_line(vday(1));
End;
```

only max 7 allowed

Script Output | Query Result

Task completed in 0.064 seconds

```
vday v_array_type := v_array_type();
Begin
     vday.extend(8);
vday(1) := 'Monday';
vday(2) := 'Tuesday';
dbms_output.put_line(vday(1));
End;
Error report -
ORA-06532: Subscript outside of limit
ORA-06512: at line 5
06532. 00000 -  "Subscript outside of limit"
```

got error

different methods

```
Declare
   Type v_array_type is varray(10) of varchar2(30);
   vday v_array_type := v_array_type(null);
Begin
   vday(1) := 'Monday';  -- assignment -- so loaded in memory
   vday.extend();
     dbms_output.put_line(vday.limit); -- max how many initialize
     dbms_output.put_line(vday.count); -- how many initialized
     dbms_output.put_line(vday.first); -- 1st .. it is always 1 -- index of first value
     dbms_output.put_line(vday.last); --  what is the last one, which is initialized -- index of last value
     vday.trim();  -- delete the last element
   --  vday.trim(2); -- last 2 element deleted
     vday.delete; -- allelements are deleted
     vday.extend(6); -- 6 elements initializes
     dbms_output.put_line(vday.count);
End;
```

Script Output | Query Result

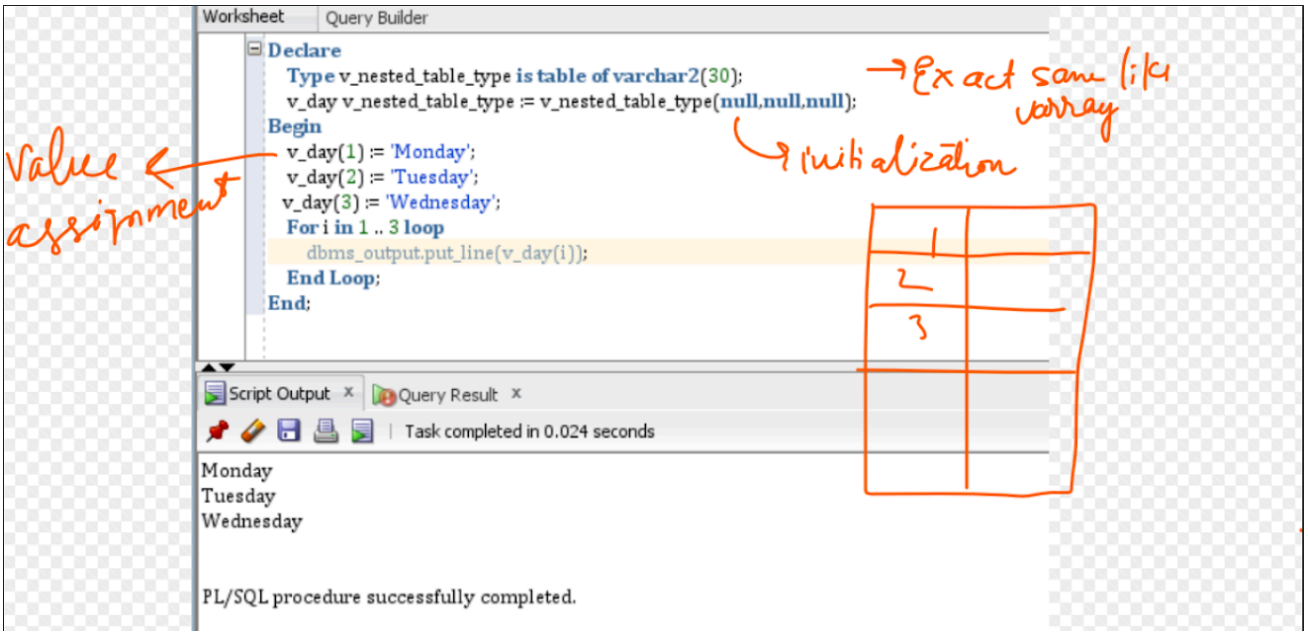
Task completed in 0.023 seconds

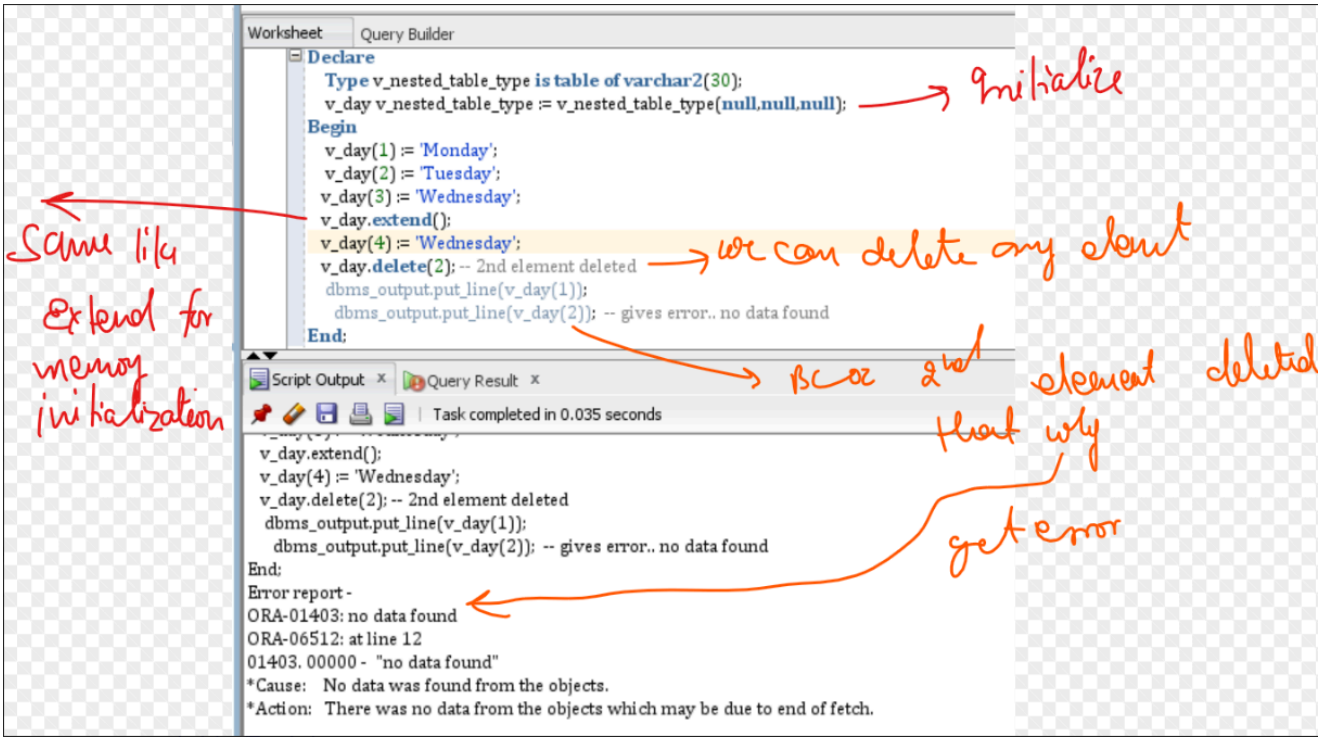
```
10
2
1
2
6


PL/SQL procedure successfully completed.
```

==========

Nested Table

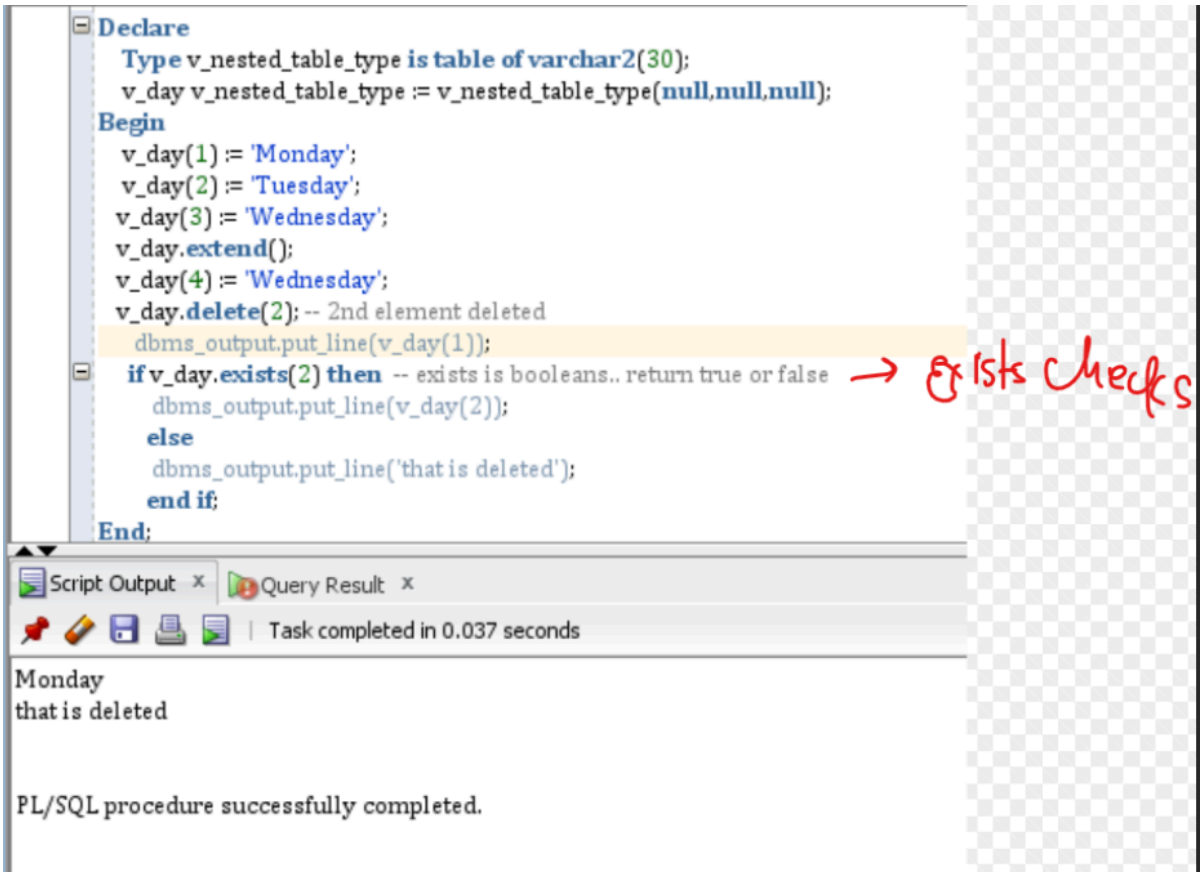
- A nested table is a type of collection that allows you to store an unordered set of elements.
- Unlike arrays or VARRAYs, which have a specific order, nested tables are unordered collections of elements. This means that you can access the elements by their position (index) within the collection, but the order of elements is not guaranteed.
- Nested tables are dynamically resizable, which means you can add or remove elements from a nested table at runtime. There is no fixed maximum size, making them suitable for scenarios where the size of the collection can change over time.
- A nested table can store elements of the same data type (homogeneous) or elements of different data types (heterogeneous)
- Varray size is fixed, but nested table can hold N number of elements . No upperbound.
- In Varray we can delete only last element or all, but here you can delete any
- Varray is continuous, but nested is no guarantee.
- **LIMIT** is only used in varray but in nested table, it returns null.





We can see in above example that unlike Varray, here we can delete any record. when we do **v_day.delete(2)**, the 2nd record from nested table is deleted. So when it tries to print, it gives error, **no_data_found**, because 2nd record does not exists. To solve these issues, we will use **EXISTS** method

```
Declare
   Type v_nested_table_type is table of varchar2(30);
   v_day v_nested_table_type := v_nested_table_type(null,null,null);
Begin
   v_day(1) := 'Monday';
   v_day(2) := 'Tuesday';
  v_day(3) := 'Wednesday';
  v_day.extend();
  v_day(4) := 'Wednesday';
  v_day.delete(2); -- 2nd element deleted
    dbms_output.put_line(v_day(1));
   if v_day.exists(2) then  -- exists is booleans.. return true or false
      dbms_output.put_line(v_day(2));
     else
      dbms_output.put_line('that is deleted');
     end if;
End;
```



==========

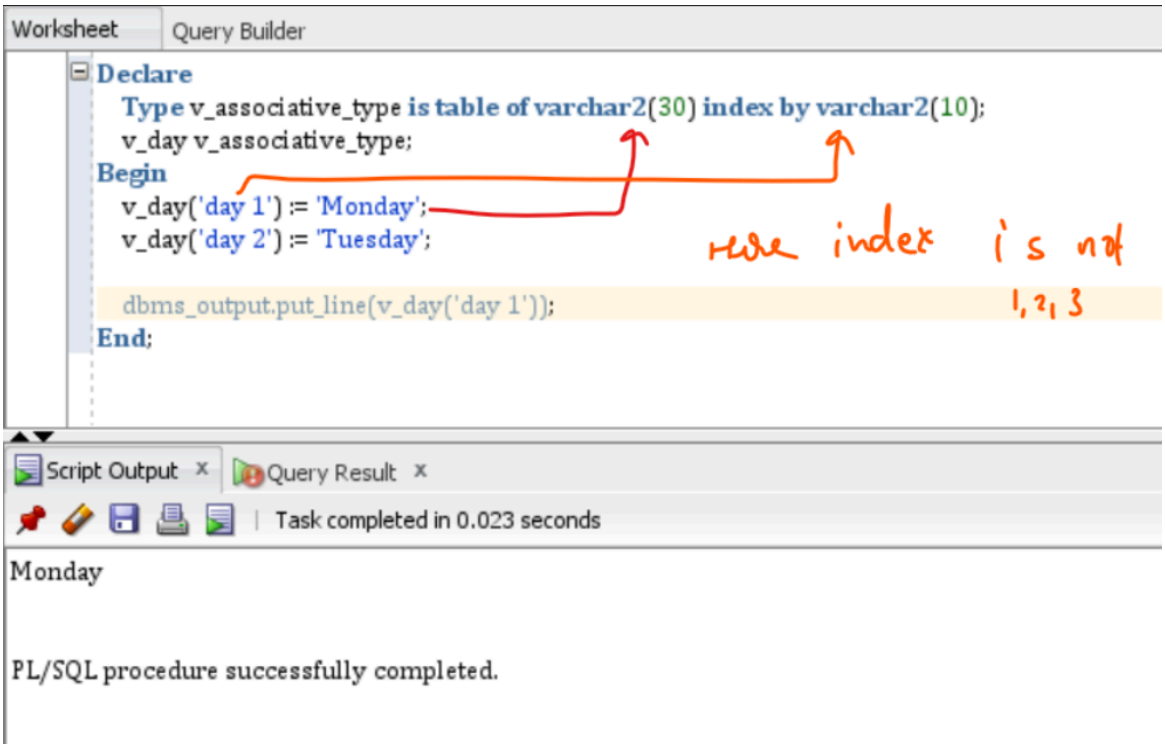
Associative array (PL/SQL Table or Index By Table)

1. associative array, also known as an index-by table or PL/SQL table.
2. Unlike nexted table and varray, it is not always 1,2,3,4 as index number. Associative arrays support two types of indexing:

- By PLS_INTEGER
- By VARCHAR2

3. The elements in an associative array are not ordered by their keys. You can access elements by specifying the key

```
DECLARE
 -- Declare an associative array type indexed by integers
 TYPE emp_salary_assoc IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
 -- Declare a variable of the emp_salary_assoc type
 emp_salaries emp_salary_assoc;
BEGIN
 -- Populate the associative array
 emp_salaries(101) := 50000;
 emp_salaries(102) := 60000;
 emp_salaries(103) := 55000;
 -- Access and display values using keys
 DBMS_OUTPUT.PUT_LINE('Employee 101 Salary: ' || emp_salaries(101));
 DBMS_OUTPUT.PUT_LINE('Employee 102 Salary: ' || emp_salaries(102));
 DBMS_OUTPUT.PUT_LINE('Employee 103 Salary: ' || emp_salaries(103));
 -- Add a new entry
 emp_salaries(104) := 58000;
 DBMS_OUTPUT.PUT_LINE('Employee 104 Salary: ' || emp_salaries(104));
 -- Remove an entry
-- emp_salaries.DELETE(102);
 DBMS_OUTPUT.PUT_LINE('Employee 102 Salary after deletion: ' || emp_salaries(102));
END;
```



Worksheet | Query Builder

```
Declare
  Type test is table of varchar2(50) index by varchar2(10);
  v_test  test;
Begin
  v_test('name') := 'vishal jaiswal';
  v_test('Address') := '23305 silcott woods ter';
  v_test('City') := 'Ashburn, Virginia';
  v_test('Zip') := '20148';
  v_test.delete('Address');
  dbms_output.put_line('value of name is '||v_test('name'));
--  dbms_output.put_line('value of Address is '||v_test('Address'));
End;
```

deleted an index

Script Output | Query Result

Task completed in 0.02 seconds

```
value of name is vishal jaiswal


PL/SQL procedure successfully completed.
```